Q1.

Consider the following piece of code!

```
...
long res=0;
...
for (i=1; i<=n; i++)
  for (j=1; j<=i; j*=2)
    res += (i+j);
printf("res=%d", res);
...
```

Figure 1A: Code for part (a)

```
...
long res=0;
...
for (i=1; i<=n; i++)
  for (j=1; j<=i; j*=2)
    res += (i+j)*(j mod i < 4);
printf("res=%d", res);
...
```

Figure 1B: Code for part (b)

a) (10pts)

What is the value of res, **in terms of *n***, printed out at the printf statement in Fig. 1A? **Show all your work!**

b) (15pts)

What is the value of res, **in terms of *n***, printed out at the printf statement in Fig. 1B? **Show all your work!**

Q2.

Consider the following structures of data:

```
struct LL_node {
      double info;                  /* 4 bytes */
      struct LL_node * next;        /* 4 bytes */
}

struct BST_node {
      double info;                  /* 4 bytes */
      struct BST_node * left;       /* 4 bytes */
      struct BST_node * right;      /* 4 bytes */
}
```

You are to decide on whether to keep *n* data nodes declared as above in a linked list (LL) or in a binary search tree (BST). The performance measure you are to use to compare the performance of a LL and a BST is defined as follows:

Performance = Total memory space allocated * Average Execution Time

or

$P=M*T.$

a.

In terms of *n*, what should be **the maximum possible execution time T_{max} of a BST** so it can outperform a LL (i.e., $P_{BST} < P_{LL}$) ?

b.

Assume that the data are presented to your program in such an order that the first 7 nodes (or less) compose a full (or a complete) binary tree and all subsequent nodes (i.e., all nodes starting from 8 up until *n*) are located on the same path in the BST. Find **the interval for the integer values of *n*** such that, for the BST explained above, $P_{BST} < P_{LL}$?

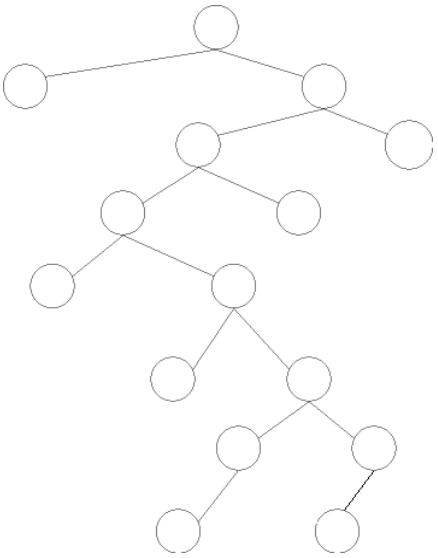
Q3.
Consider the arithmetic string below:

3 1 + 2 3 2 ^ a 2 / 4 3 - b - c * 2 4 1 + ^ 3 4 * 2 2 * - / 6 2 * - + = 0

where a, b and c are arithmetic operators or positive integers. Assuming the results of all subsequent operations turn out integers, find (at least one valid triple of) a, b and c!

Show all your work!

Q4.
S = {8, 19, 28, 41, 48, 59, 67, 69, 70, 73, 77, 80, 88, 98, 101}



Consider the set of numbers *S* as given above (i.e., above is only shown what elements there exist in *S*. The sequence of numbers above is not important). Initially, this set consists of 15 integers.

a)
Fill all numbers in the above binary tree topology such that it becomes a binary search tree (BST).

b)
What is the **worst case, best case, and average case search times of the above BST**, assuming that a comparison of the searched item with the key of a tree node is 1 time unit?

c)
Assuming no key duplication, what is the **highest number of nodes** that can be added to a subtree whose root is the right child of the node with key 88 without relocating (i.e., changing the places of) existing keys? Please **provide the set of numbers that you can add**!

d)
In how many different ways can you insert all nodes in part (c), provided that

- $t_{av} \leq 5.75$, and
- $t_{wc}^{(d)} \leq t_{wc}^{(b)} + 1$

$t_{wc}^{(b)}$ is the worst case search time in part (b) of the question

$t_{wc}^{(d)}$ is the worst case search time in part (d) of the question